

レポートタイトル

著者: あなたの名前
日付: 2026年4月23日

本文スタート

課題 (1)

prog1-1.cのプログラムの実行結果において、変数や構造体の間のパディング、各バイトごとの解釈について考察する。

まず、実行結果からアドレスの小さい方からf, name2, d, e, name1という順にメモリ上に書かれていることが確認できた。これは関数内で宣言された変数がメモリのスタック状に確保していることに起因すると考えられる。コード上ではname1, d, e, f, name2の順に宣言されているが、大きなアドレスから小さなアドレスに向かって下向きに成長していくため、最初に宣言されたname1が最も大きなアドレスに置かれ、後に宣言された変数順に小さなアドレスへと積み上げられたものと考えられる。

一方で、構造体dやeの内部にあるデータや配列の並び順については、これとは別のルールが適用されている。構造体dの内部ではソースコード上で先に宣言されたa, b, cの順にアドレスが大きくなっており、構造体eの内部でも同様に宣言順であるa, c, bの順に配置されていることが確認できる。

以上のように、プログラム内で宣言された変数の大まかな配置順序は、スタックによってメモリ上に確保され、アドレスの下方向に成長する特徴を持つことが分かった。しかしそれとは別に、構造体などの内部における細かなデータの並び順は、プログラミング言語側の仕様によってメモリアドレスが大きくなる方向へ順番に配置するよう厳密に定められていることがわかった。

次に、データとデータの間のパディングについて述べる。構造体dにおいて、d.bとd.cの間に1バイトの隙間が生じている。これは、char型のd.aで2バイト、d.bで1バイトの合計3バイトを使った後、次に配置されるd.cが4バイトの整数型であるため、d.cの開始位置を4の倍数のアドレスに揃えるため1バイトの隙間が挿入されたと考えられる。これと同様に、構造体eにおいては、e.aの2バイトの後に4バイトの整数型e.cが来るため、4の倍数のアドレスに合わせるため2バイトのパディングが挿入されている。このように、構造体の中身はコンピュータが読みこむ際に処理効率を上げるため、位置の整列させるための微調整が行われている。一方、ポインタ変数name2と構造体dの間に空いた4バイトや、構造体eの末尾と配列name1の間に空いた4バイトのような、変数と変数の間に生じた隙間も、同じような整列方法がとられている。メモリ上に変数の領域を確保していく際、変数全体の開始位置を8バイトや16バイトといったより大きな区切りに合わせようとする仕様が確認できた。

次に、メモリ上に敷き詰められた各バイトの数値がどのように解釈されているかについて考察する。文字を扱うname1やe.aに格納されたデータは、文字に対応した1バイトのASCIIコードの数値としてそのまま記録されている。一方、数値を扱う部分では、d.cの-1はffffff、d.aの-9はf7のように記録される。さらに、int型の整数の場合は、下の位のデータから順番にメモリに書き込んでいく方式がとられている。たとえば、e.cの-2147483648という値は、16進数に直すと80000000となるが、メモリ上には下の位から順に00,00,00,80と記録される。

課題 (2)

出力結果

-1, -1
0, 256
1, 9

(1) (2)の値

(1) 255 (2) 2305

メモリマップ

Pointer	Adress	Value	Variable
	0x7fff8b149218	23	bi
	19	92	bi
	1a	14	bi
	1b	8b	bi
	1c	ff	bi
	1d	7f	bi
	1e	00	bi
	1f	00	bi
&ai[0]	0x7fff8b149220	f7	ai[0]
&ai[0]	21	ff	ai[0]
&ai[1]	22	01	ai[1]
&ai[1]	23	00	ai[1]
&ai[2]	24	01	ai[2]
&ai[2]	25	00	ai[2]
&ai[3]	26	00	ai[3]
&ai[3]	27	80	ai[3]

biの中には0x7fff8b149223というアドレス値が保存されている。この値を配列aiのアドレス列と照らし合わせると、このアドレスは配列ai[1]の2バイト目の場所を指し示すことがわかる。配列aiの先頭アドレスは末尾20であるため、そこからプラス3バイトずれた位置であり、biは「先頭から3バイト目の位置を指す」と考えられる。実際のコード内では、ポインタアドレスbiは以下のように宣言されている。

short *bi = (short *)(((int *)((char *)ai - 1) + 1) - sizeof(ai)/sizeof(int) + 2;

以上のbiの宣言文において、まず `(char *)ai - 1` はaiの先頭アドレスから-1バイトの位置を指す。これをint型のキャストして1を足すと、すなわちint型分の4バイト進むことになる。この時点で、aiの先頭から3バイト目 ($-1 + 4 = 3$) を指している。次に、`sizeof(ai)/sizeof(int)` の部分にて、ai全体のサイズをint型のサイズで割るため、 $8 \div 4 = 2$ となる。式全体として最後に2が足され、結果としてポインタbiは配列aiの先頭から「3バイト目」のアドレスを指すshort型ポインタとして機能していることがわかった。

出力結果からのメモリ配置の解析について述べる。C言語において、配列はインデックスが大きくなるにつれてメモリアドレスも大きくなる方向に配置される。表を参照し、配列aiが確保した8バイトのメモリ領域を、先頭から順番に [20] から [27] と表記する。先のポインタ演算の解析により、ポインタbiは配列aiの先頭から3バイト目、すなわち [23] を起点として扱っていることがわかっている。

これを踏まえてループによる出力結果を順に読み解いていく。forループのインデックスが-1から始まっていることに着目する。biの起点が [23] であるため、一つ前の要素であるbi[-1]が参照するメモリは [21] と [22] になる。2バイトのshort型においてマイナス1は16進数で0xFFFFと表現されるため、[21] と [22] にはそれぞれ0xFFが格納されていることがわかる。次に、bi[0]の出力結果である256について考える。これが参照するメモリは起点そのものである [23] と [24] である。256は16進数で0x0100となるが、リトルエンディアン方式に従って下位バイトから順にメモリに配置されるため、[23] には0x00が、[24] には0x01が格納されることになる。同様に、bi[1]の出力結果である9については、[25] と [26] の領域を参照する。9は16進数で0x0009であり、これも下位バイトから順に配置されるため、[25] には0x09が、[26] には0x00が格納されていると導き出せる。

こうして特定された [21] から [26] までの1バイトごとのデータを用いて、元の配列aiの未知の要素を復元していく。プログラム中の値(1)にあたるai[1]は、メモリ上の [22] と [23] の2バイトから構成されている。これまでの解析から [22] は0xFF、[23] は0x00であることが判明しているため、上位バイトである [23] と下位バイトである [22] を本来の順序に合わせて16進数のデータに戻すと0x00FFとなる。これを10進数に変換すると255となり、これが値(1)の正解である。続いて、値(2)にあたるai[2]は [24] と [25] から構成されている。[24] は0x01、[25] は0x09であるため、同様に16進数に戻すと0x0901となる。これを10進数に変換すると2305となり、これが値(2)の正解となる。